# Дорожная карта: CLI-трекер расходов (dev.house, неделя 1)

> Этот файл только для тебя. **Не коммить его** ( `git add` делай по имени файла: `git add week1/homework.py` ). Каждый пример кода проверен на Python 3.14 и твоём `week1/expenses.csv`. Пиши код сам руками: так ты его поймёшь.

Срок сдачи: 27 октября. Всё ниже занимает около 1–1,5 часа.

---

## 0. Что мы строим

```
python week1/homework.py week1/expenses.csv
```

Программа читает CSV `дата,категория,сумма` и печатает суммы по категориям и по месяцам. Если файла нет, он пустой или в нём плохие данные, она пишет понятную строку ошибки, а не traceback.

### Цепочка функций (кто кого вызывает)

```
main()
 ├─ parse_args()                 → что ввёл пользователь в командной строке
 ├─ read_rows(path)              → читает файл, отдаёт список словарей (всё строки)
 ├─ load_expenses(rows)          → список кортежей (date, str, float)
 │    └─ parse_row(row, line)    → один словарь → один кортеж, проверяет данные
 └─ print_report(expenses)       → печатает результат
      ├─ group_sum(expenses, category_of / month_of)   → dict с суммами
      └─ format_totals(title, totals)                  → красивый текст
```

Данные по пути проходят такие превращения:

```
файл (текст)
  → list[dict[str, str]]        (read_rows: всё ещё строки!)
  → list[tuple[date,str,float]] (load_expenses: настоящие типы)
  → dict[str, float]            (group_sum: суммы)
  → str                         (format_totals: текст для печати)
```

### Требования задания и где они закрываются

| Требование | Где у нас |
|---|---|
| Путь из аргумента командной строки | `parse_args` ( `argparse` ) |
| `list` | список расходов `expenses` , список словарей `rows` |
| `tuple` | один расход `(date, str, float)` |
| `set` | множество уникальных категорий `{...}` (шаг 7) |
| `dict` | строка CSV как словарь; итоговые суммы |
| Минимум одна comprehension | `load_expenses` , `format_totals` , множество категорий |
| Функция высшего порядка или замыкание | `group_sum(expenses, key)` ; бонус: замыкание (раздел 12) |
| Аннотации типов у всех функций | везде `->` и `: тип` |
| Ошибки без traceback | `InputError` + `try/except` в `main` |
| README с запуском и выводом | шаг 9 |
| История коммитов по шагам | шаг 10 |

---

## 1. Что у тебя уже есть и что поправить сразу

В `week1/homework.py` уже есть `parse_args` , `read_rows` , `parse_row` , класс `InputError` . Проверь и поправь:

- [ ] `add_arguemtn` → `add_argument` (опечатка, будет `AttributeError` )
- [ ] убери `print(reader.fieldnames)` из `read_rows`
- [ ] пустые `elif` / `if` без тела это `SyntaxError` : пока закомментируй `run_menu` целиком
- [ ] `main` должна быть `-> int` , а в конце файла `sys.exit(main())`
- [ ] `load_extenses` → `load_expenses` (опечатка), и заменить тело (шаг 4)
- [ ] `week1/expenses_big.csv` git видит как удалённый. Если не удалял сам: `git restore week1/expenses_big.csv` . Рабочий файл сейчас `week1/expenses.csv` (34 строки данных).

Запуск всегда из корня репозитория:

```
python week1/homework.py week1/expenses.csv
```

---

## 2. Шаг 1 — `parse_args`: что ввёл пользователь

### Как это работает под капотом

Когда ты запускаешь `python homework.py file.csv --top 5`, Python кладёт слова после имени скрипта в список `sys.argv`:

```
sys.argv == ["homework.py", "file.csv", "--top", "5"]    # все элементы — строки
```

`argparse` берёт этот список, разбирает его по правилам, которые ты описал, преобразует типы и возвращает объект `Namespace` с атрибутами: `args.path`, `args.top`. Если аргументов не хватает, он сам печатает usage-сообщение и завершает программу с кодом 2. Поэтому проверка `if args.path is None` не нужна.

### Код

```
def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(description="Трекер расходов")
    parser.add_argument("path", type=Path, help="CSV: date,category,amount")
    parser.add_argument("--top", type=int, default=3, help="сколько топ-категорий
показать")
    return parser.parse_args()
```

- `"path"` без дефисов — **позиционный** аргумент, обязательный.
- `"--top"` с дефисами — **опциональный** флаг, у него есть `default`.
- `type=Path` превращает строку в `Path`, `type=int` в число. Если введут `--top abc`, `argparse` сам выдаст понятную ошибку.
- Весь код парсера **внутри** функции. Иначе он выполнится уже при `import homework`.

### Другие варианты

| Вариант | Плюсы / минусы |
|---|---|
| `sys.argv` вручную | Допустим по заданию. Но проверки, `--help` и типы пишешь сам. |
| `argparse` (выбираем) | В стандартной библиотеке, даёт `--help`, типы, сообщения об ошибках. |
| `click`, `typer` | Удобнее, но это сторонние библиотеки, лишнее для домашки. |

---

## 3. Шаг 2 — `read_rows`: читаем файл

### Как это работает под капотом

- `open(path, ...)` просит операционную систему открыть файл и возвращает файловый объект `f`.

- `with ... as f:` гарантирует, что файл будет закрыт, даже если внутри случится ошибка.
- `csv.DictReader(f)` — **итератор** (объект класса `csv.DictReader`). Он ленивый: читает файл по одной строке, когда у него просят следующую. Первая строка файла становится списком ключей `reader.fieldnames`, каждая следующая превращается в `dict`.
- Итератор **одноразовый**: после `list(reader)` второй `list(reader)` вернёт `[]`. И он работает только пока файл открыт, поэтому `return list(reader)` стоит **внутри** `with`.
- Все значения в словаре это **строки**, даже `'2500.00'`. CSV не знает типов.
- `encoding="utf-8-sig"` читает UTF-8 и убирает невидимую метку BOM (её добавляет Excel), иначе первый ключ был бы `'date'`.
- `newline=""` советует документация `csv`, чтобы переносы внутри ячеек не ломались.

### Код

```
def read_rows(path: Path) -> list[dict[str, str]]:
    if path.suffix.lower() != ".csv":
        raise InputError("нужен файл с расширением .csv")
    with open(path, encoding="utf-8-sig", newline="") as f:
        reader = csv.DictReader(f)
        fields = reader.fieldnames
        if fields is None:
            raise InputError("файл пустой")
        missing = {"date", "category", "amount"} - set(fields)
        if missing:
            raise InputError(f"нет колонок: {', '.join(sorted(missing))}")
        return list(reader)
```

- `path.suffix` — расширение вместе с точкой (`.csv`), регистр сохраняется, поэтому `.lower()`.
- У пустого файла `fieldnames` равен `None`. Это и есть «пустой файл».
- `{"date","category","amount"} - set(fields)` — **разность множеств**: что из нужного отсутствует в файле. Так мы честно используем `set`.

### Как сообщать об ошибках: print, return или exit?

| Способ | Что даёт | Минус |
|---|---|---|
| `print` + `sys.exit(1)` прямо в функции | Просто | Функцию нельзя переиспользовать и проверить: она убивает всю программу |
| `return None` или код ошибки | Просто | Каждый вызывающий должен не забыть проверить |
| **`raise InputError(...)`** (выбираем) | Ошибка летит вверх сама, ловим в одном месте | Надо знать про `try/except` |

### Другие способы читать CSV

| Вариант | Комментарий |
|---|---|
| `csv.DictReader` (выбираем) | Ключи по именам колонок, понятный код |
| `csv.reader` | Отдаёт списки, надо помнить номера колонок (`row[2]`) |
| `pandas.read_csv` | Мощно, но тяжёлая зависимость и скрывает суть |
| `open().read().split("\n")` | Сломается на запятых в кавычках: «Кафе, обед» |

### Проверка типа файла

Мы смотрим только расширение (быстро). Альтернативы: `mimetypes` (тоже по расширению), чтение первых байт «магические числа» (у CSV их нет), или просто попробовать прочитать и поймать `UnicodeDecodeError`. Мы делаем и первое, и последнее: расширение проверяем сразу, а бинарный файл поймаем в `main`.

---

## 4. Шаг 3 — `parse_row`: одна строка → один расход

### Зачем

`read_rows` отдаёт словари со строками, считать их нельзя. `parse_row` берёт **один** словарь и возвращает **один** кортеж `(date, str, float)` с настоящими типами. Список всех расходов собирает другая функция (шаг 4).

### Что внутри

- `date.fromisoformat("2026-01-05")` → `date(2026, 1, 5)`. Неверная дата (`2026-02-30`, `05.01.2026`) → `ValueError`. Получив `date`, потом легко брать месяц: `d.strftime("%Y-%m")` → `"2026-01"`.
- `float("2500.00")` → `2500.0`. `float("abc")` и `float("12,5")` → `ValueError`. Если в строке CSV не хватило колонки, значение будет `None`, и `float(None)` даст `TypeError`. Нет ключа в словаре → `KeyError`. Поэтому ловим все три.
- Опасная ловушка: `float("nan")` **не** ошибка, а `nan <= 0` равно `False`, и такая сумма проскочит проверку. Поэтому используем `math.isfinite` (оно отсекает `nan` и `inf`).
- `line` — номер строки в файле, нужен **только** для текста ошибки. Откуда он берётся, смотри шаг 4.
- Кортеж неизменяемый (`t[2] = 1` → `TypeError`). Для «записи о расходе» это хорошо: её не испортят случайно.
- `from None` в `raise` скрывает внутреннюю ошибку и оставляет только наше сообщение.

### Код

```
def parse_row(row: dict[str, str], line: int) -> Expense:
    try:
        d = date.fromisoformat(row["date"])
        cat = row["category"].strip()
        amount = float(row["amount"])
    except (ValueError, KeyError, TypeError):
        raise InputError(f"строка {line}: неверные данные") from None
    if not math.isfinite(amount) or amount <= 0:
        raise InputError(f"строка {line}: сумма должна быть положительным числом")
    if not cat:
        raise InputError(f"строка {line}: пустая категория")
    return (d, cat, amount)
```

Не забудь `import math` в начале файла. `Expense = tuple[date, str, float]` у тебя уже есть: это просто подпись типа, вызывать её как класс нельзя (`Expense(d, c, a)` даст `TypeError`).

### Почему не просто `except:`

Голый `except:` ловит всё, даже `Ctrl+C` и настоящие баги. Перечисляй только те исключения, которых ждёшь.

### Другие способы хранить расход

| Вариант | Комментарий |
|---|---|
| `tuple` (выбираем) | Задание просит `tuple`; неизменяемый |
| `NamedTuple` / `dataclass` | Читабельнее (`e.amount` вместо `e[2]`), но это материал недели 2 |
| `dict` | Гибко, но легко опечататься в ключе |

---

## 5. Шаг 4 — `load_expenses`: все строки сразу

### Как это работает

`enumerate(rows, start=2)` выдаёт пары `(номер, элемент)`: `(2, первый_словарь)`, `(3, второй)`, … Заголовок стоит на строке 1, значит первые данные на строке 2. Это и есть тот самый `line`, который попадает в `parse_row`. Если `parse_row` бросит `InputError`, он «пролетит» через эту функцию сам, ловить его здесь не нужно.

Квадратные скобки с `for` внутри это **list comprehension**: сокращённая запись цикла, который собирает список.

```
# так же, как
result = []
for i, row in enumerate(rows, start=2):
    result.append(parse_row(row, i))
```

### Код

```
def load_expenses(rows: list[dict[str, str]]) -> list[Expense]:
    if not rows:
        raise InputError("в файле нет данных")
    return [parse_row(row, i) for i, row in enumerate(rows, start=2)]
```

Пустой список это случай «в файле только заголовок».

> Важно: в `load_expenses` **не** копируй проверку даты и суммы. Одна проверка живёт в одном месте (`parse_row`).

---

## 6. Шаг 5 — ключи и `group_sum`: функция высшего порядка

### Идея

Нужно считать суммы по категориям и по месяцам. Код одинаковый, отличается только то, **по чему** группируем. Поэтому «по чему» передаётся как параметр, и это параметр-функция.

В Python функции — обычные значения: их можно положить в переменную, передать в другую функцию, вернуть из функции.

### Код

```
def category_of(e: Expense) -> str:
    return e[1]

def month_of(e: Expense) -> str:
    return e[0].strftime("%Y-%m")

def group_sum(expenses: list[Expense], key: Callable[[Expense], str]) -> dict[str, float]:
    totals: dict[str, float] = {}
    for e in expenses:
        k = key(e)
        totals[k] = totals.get(k, 0.0) + e[2]
    return totals
```

Импорт: `from collections.abc import Callable`.

### Пошаговый разбор на трёх расходах

```
expenses = [(date(2026,1,5),"Еда",2500.0), (date(2026,1,9),"Транспорт",1000.0),
(date(2026,2,3),"Еда",500.0)]
group_sum(expenses, category_of)
```

| Шаг | Расход | `key(e)` | `totals` после |
|---|---|---|---|
| 1 | Еда, 2500 | `"Еда"` | `{'Еда': 2500.0}` |
| 2 | Транспорт, 1000 | `"Транспорт"` | `{'Еда': 2500.0, 'Транспорт': 1000.0}` |
| 3 | Еда, 500 | `"Еда"` | `{'Еда': 3000.0, 'Транспорт': 1000.0}` |

С `month_of` тот же цикл даст `{'2026-01': 3500.0, '2026-02': 500.0}`.

### Ключевые детали

- Индексы кортежа с нуля: `e[0]` дата, `e[1]` категория, `e[2]` сумма.
- `totals.get(k, 0.0)` = «значение по ключу, а если ключа нет, то 0.0».
- Вызов `group_sum(expenses, category_of)`: функцию передаём **без скобок**. `category_of` это сама функция, а `category_of(e)` уже её результат.
- Функцию `key` вызывает сам `group_sum` внутри, строкой `k = key(e)`.
- `Callable[[Expense], str]` читается «функция, которая принимает один `Expense` и возвращает `str`». Python аннотации не проверяет, они для читателя и редактора.

### Другие варианты

```
# 1) defaultdict: ключа нет → создаётся 0.0 сам
from collections import defaultdict
totals = defaultdict(float)
totals[k] += e[2]

# 2) Counter: специализирован на подсчёте, поддерживает суммы
from collections import Counter
totals = Counter()
totals[k] += e[2]

# 3) comprehension + вложенный цикл: короче, но обходит список по разу на ключ (медленнее)
{k: sum(e[2] for e in expenses if key(e) == k) for k in {key(e) for e in expenses}}
```

Мы выбираем `dict` + `.get`: он самый прозрачный и не требует новых импортов.

---

## 7. Шаг 6 — `format_totals`: красивый вывод

```
def format_totals(title: str, totals: dict[str, float]) -> str:
    lines = [f"{k}: {v:.2f}" for k, v in sorted(totals.items())]
    return title + "\n" + "\n".join(lines)
```

- `totals.items()` отдаёт пары `(ключ, значение)`, `sorted(...)` сортирует по ключу (русские названия, даты `2026-01`... сортируются как надо).
- `f"{v:.2f}"` — число с двумя знаками после запятой: `11400.0` → `11400.00`.
- `"\n".join(lines)` склеивает строки через перенос.
- Это вторая comprehension в программе.

Проверка:

```
print(format_totals("По категориям:", group_sum(expenses, category_of)))
```

---

## 8. Шаг 7 — `print_report`: собираем отчёт (пишешь сам по описанию)

Функция печатает:

1. блок «По категориям:» (через `group_sum` + `category_of` + `format_totals`);
2. пустую строку;
3. блок «По месяцам:» (то же с `month_of`);
4. по желанию: число уникальных категорий и топ-N.

Каркас:

```
def print_report(expenses: list[Expense], top: int) -> None:
    by_cat = group_sum(expenses, ...)
    by_month = group_sum(expenses, ...)
    categories = {... for e in expenses}      # set comprehension: фигурные скобки, без пар
key: value
    print(...)
    print()
    print(...)
    print(f"Уникальных категорий: {len(categories)}")
```

- **`set`** — коллекция уникальных значений. `{category_of(e) for e in expenses}` — set comprehension. Дубликаты («Еда» встречается много раз) схлопываются сами. Это четвёртая обязательная коллекция.
- **Топ-N по сумме** (по желанию, использует `--top`):

```
def top_categories(totals: dict[str, float], n: int) -> list[tuple[str, float]]:
    return sorted(totals.items(), key=lambda kv: kv[1], reverse=True)[:n]
```

`key=lambda kv: kv[1]` — маленькая безымянная функция: «возьми у пары второй элемент (сумму)». Ещё одна функция высшего порядка (`sorted` принимает функцию). `reverse=True` — по убыванию, `[:n]` — срез первых n.

Ожидаемый вывод на `week1/expenses.csv` (то, что должно получиться, сверь):

```
По категориям:
Аренда: 170000.00
Еда: 11400.00
Кафе: 15700.00
Обучение: 19500.00
Продукты: 61100.00
Развлечения: 11500.00
Связь: 14000.00
Транспорт: 7800.00

По месяцам:
2026-01: 26700.00
2026-02: 37900.00
2026-03: 124900.00
2026-04: 121500.00
```

Эти числа совпадают с примером из README стартера (`cli_starter/expense_tracker/README.md`). Если у тебя то же самое, логика верная.

---

## 9. Шаг 8 — `main`: порядок и обработка ошибок (пишешь сам по описанию)

### Как ошибки идут вверх

```
parse_row   бросает InputError ──┐
load_expenses (ничего не ловит) ─┤  летит вверх
read_rows   бросает InputError ──┤
                                 ▼
main:  try: ... except InputError → печать в stderr → return 1
```

`main` — единственное место, где ошибки превращаются в текст для человека. Остальные функции только бросают.

### Что ловить в `main`

| Исключение | Когда |
|---|---|
| `InputError` | наша: не .csv, пустой файл, плохая строка, нет колонок |
| `OSError` | файла нет ( `FileNotFoundError` — его подкласс), это папка, нет прав |
| `UnicodeDecodeError` | бинарный файл, не UTF-8 (это подкласс `ValueError`, не `OSError`, поэтому отдельный `except`) |

### Каркас

```
def main() -> int:
    args = parse_args()
    try:
        expenses = load_expenses(read_rows(args.path))
    except InputError as e:
        print(f"Ошибка: {e}", file=sys.stderr)
        return 1
    except UnicodeDecodeError:
        print("Ошибка: файл не в кодировке UTF-8", file=sys.stderr)
        return 1
    except OSError as e:
        print(f"Ошибка: не удалось прочитать файл ({e.strerror})", file=sys.stderr)
        return 1
    print_report(expenses, args.top)
    return 0

if __name__ == "__main__":
    sys.exit(main())
```

- `file=sys.stderr` пишет в поток ошибок, а не в обычный вывод (отчёт можно перенаправить в файл, ошибки останутся на экране).
- `return 1` и `sys.exit(main())`: код выхода 0 = успех, не 0 = ошибка. Так принято в командной строке, это же использует CI.
- Порядок `except` важен: `UnicodeDecodeError` и `OSError` не пересекаются, но `InputError` лучше первым.

### Проверка (все четыре случая должны быть без traceback)

```
python week1/homework.py week1/expenses.csv      # отчёт
python week1/homework.py nope.csv                # Ошибка: не удалось прочитать файл (No
such file or directory)
python week1/homework.py week1                   # Ошибка: нужен файл с расширением .csv
(это папка)
python week1/homework.py week1/NOTES.md          # Ошибка: нужен файл с расширением .csv
```

Ещё проверь вручную: пустой файл (создай `empty.csv` без содержимого) → «файл пустой»; файл только с заголовком → «в файле нет данных»; строка с датой `2026-02-30` → «строка N: неверные данные»; сумма `abc` → то же; сумма `nan` → «сумма должна быть положительным числом».

---

## 10. Шаг 9 — README.md

Создай `week1/README.md` (или в корне). Что должно быть (по критериям приёмки):

````
# Трекер расходов

Консольная утилита: читает CSV `дата,категория,сумма` и показывает суммы по категориям и месяцам.

## Требования
Python 3.12+

## Запуск
```bash
python week1/homework.py week1/expenses.csv
python week1/homework.py week1/expenses.csv --top 5
```

## Формат CSV
Заголовок `date,category,amount`, дата `YYYY-MM-DD`, сумма с точкой, кодировка UTF-8.

## Пример вывода
(вставь РЕАЛЬНЫЙ вывод своей программы)

## Ошибки
Нет файла, не .csv, пустой файл, плохая строка → короткое сообщение и код выхода 1, без traceback.

## Что внутри
list, tuple, set, dict, comprehension, функция высшего порядка (group_sum), аннотации типов.
````

Пример вывода копируй из терминала после запуска, не выдумывай.

---

## 11. Шаг 10 — git: коммиты по шагам

Задание: «не один коммит done, а шаги работы». У тебя уже есть 2 коммита. Делай коммит после каждого рабочего шага:

```
git add week1/homework.py
git commit -m "add argument parsing and fix csv reading"

git add week1/homework.py
git commit -m "add parse_row and load_expenses with validation"

git add week1/homework.py
git commit -m "calculate totals by category and month with group_sum"

git add week1/homework.py
git commit -m "add report output and error handling"

git add week1/README.md
git commit -m "document usage"

git push
```

- `git status` перед каждым коммитом: убедись, что в списке только нужное (ROADMAP.md не добавляй).
- Если файлов `expenses.csv` и `expenses_big.csv` в `git status` что-то странное, реши, нужен ли данные в репозитории: для ментора лучше добавить `week1/expenses.csv` (`git add week1/expenses.csv`), чтобы команда из README работала.
- Репозиторий должен быть публичным или открытым для ментора. Remote у тебя уже есть: `https://github.com/Kair97/DevPython-KOZ-97`.

---

## 12. Бонусы (только если останется время)

### Замыкание (второй вариант для пункта «функция высшего порядка или замыкание»)

Замыкание — функция, которая запоминает значение из внешней функции:

```
def make_min_filter(minimum: float) -> Callable[[Expense], bool]:
    def check(e: Expense) -> bool:
        return e[2] >= minimum
    return check

big = make_min_filter(10000)
only_big = [e for e in expenses if big(e)]
```

`check` «помнит» `minimum`, хотя `make_min_filter` уже закончила работу. Можно сделать флаг `--min 10000`.

### Интерактивный режим ( `--interactive` )

Флаг `action="store_true"` (по умолчанию `False`, при указании `True`). Если он включён, вызываешь `run_menu(expenses)` с циклом `while True`: печатаешь меню, читаешь `input("> ")`, по выбору показываешь отчёт, на `0` выходишь через `break`. Оберни цикл в `try/except (KeyboardInterrupt, EOFError)`, чтобы `Ctrl+C` не давал traceback. Читай файл **один раз до** цикла.

### Другие форматы даты

`datetime.strptime("05.01.2026", "%d.%m.%Y").date()`. Коды: `%Y` год, `%m` месяц, `%d` день, `%B` название месяца. Для задания хватает `fromisoformat`.

---

## 13. Текст для LMS (шаблон)

```
Сделал трекер расходов.
Репозиторий: <ссылка>
Запуск: python week1/homework.py week1/expenses.csv   (опционально --top 5)
Что сделано: чтение CSV через argparse + csv.DictReader, проверка данных,
суммы по категориям и месяцам (group_sum - функция высшего порядка),
list/tuple/set/dict, comprehension, аннотации, ошибки без traceback.
README с примером вывода, коммиты по шагам.
Вопросы / что не получилось: <честно напиши, если что-то не сделал, например меню>
```

---

## 14. Частые ошибки и что они значат

| Что видишь | Причина | Что делать |
|---|---|---|
| `SyntaxError` на пустом `if/elif` | у блока нет тела | поставь `...` или `pass` |
| `ValueError: I/O operation on closed file` | `list(reader)` после `with` | сдвинь `return` внутрь `with` |
| `AttributeError: ... has no attribute 'add_arguemtn'` | опечатка | `add_argument` |
| `TypeError: tuple expected at most 1 argument, got 3` | вызвал `Expense(d, c, a)` | верни `(d, c, a)` |
| `TypeError: parse_row() missing 1 required positional argument` | не передал `line` | `parse_row(row, i)` |
| `KeyError: 'date'` | в словаре нет такого ключа (BOM или `fieldnames=`) | `utf-8-sig`, убрать свои `fieldnames` |
| `UnicodeDecodeError` | бинарный файл или не UTF-8 | ловим в `main` |
| `FileNotFoundError` | неверный путь | запускай из корня, путь `week1/expenses.csv` |
| Пустой вывод | список `rows` пустой или `print` не вызван | проверь `print(len(rows))` |

## 15. Чек-лист перед сдачей

- [ ] `python week1/homework.py week1/expenses.csv` печатает ожидаемый отчёт
- [ ] Нет файла / папка / не .csv / пустой / плохая строка → одна строка ошибки, без traceback
- [ ] В коде есть `list`, `tuple`, `set`, `dict`
- [ ] Есть comprehension и функция высшего порядка (`group_sum`)
- [ ] У каждой функции есть аннотации параметров и возвращаемого значения
- [ ] `README.md` с командой запуска и реальным выводом
- [ ] Коммиты по шагам, `git push` сделан
- [ ] Ссылка на репозиторий и текст на LMS отправлены (срок 27 октября)